4. BÚSQUEDA ENTRE ADVERSARIOS
(PARTE 2)

IA 3.2 - Programación III

1° C - 2026

Dr. Mauro Lucci

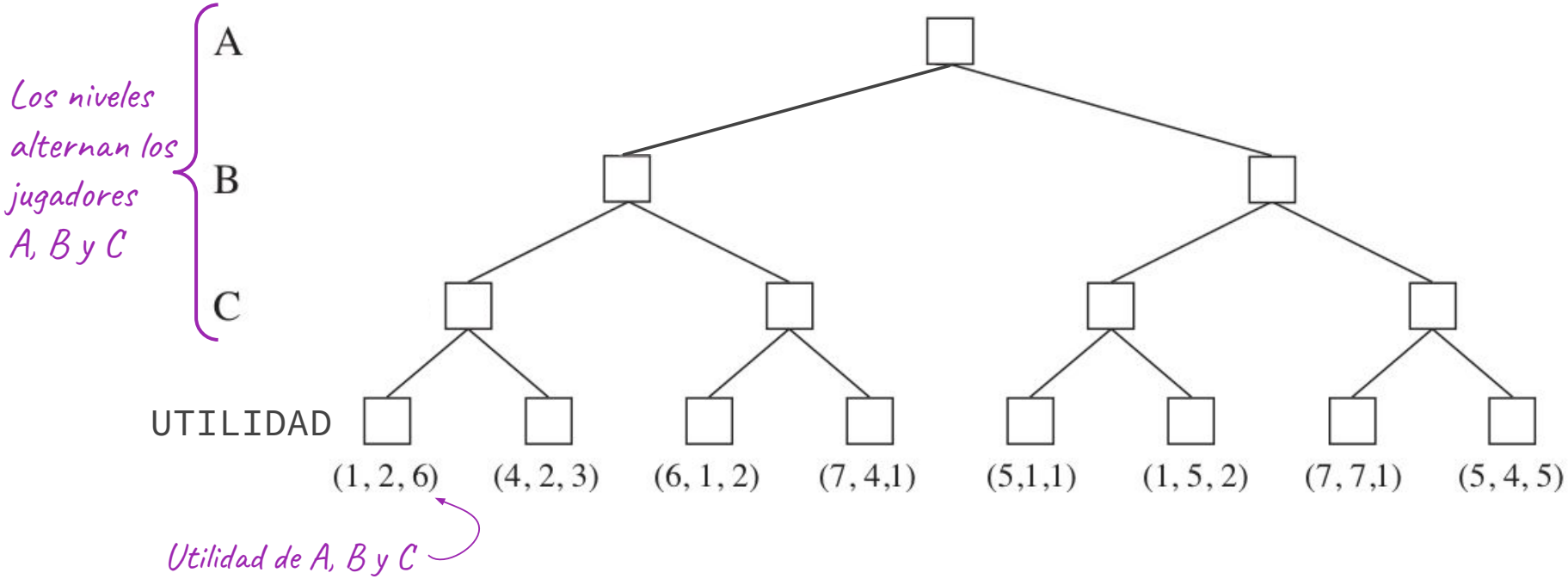
Juegos multijugadores

Son juegos con **más de dos** jugadores.

En los estados terminales, hay un valor de utilidad **por cada jugador**.

O bien, la función utilidad toma un estado terminal y devuelve un **vector**.

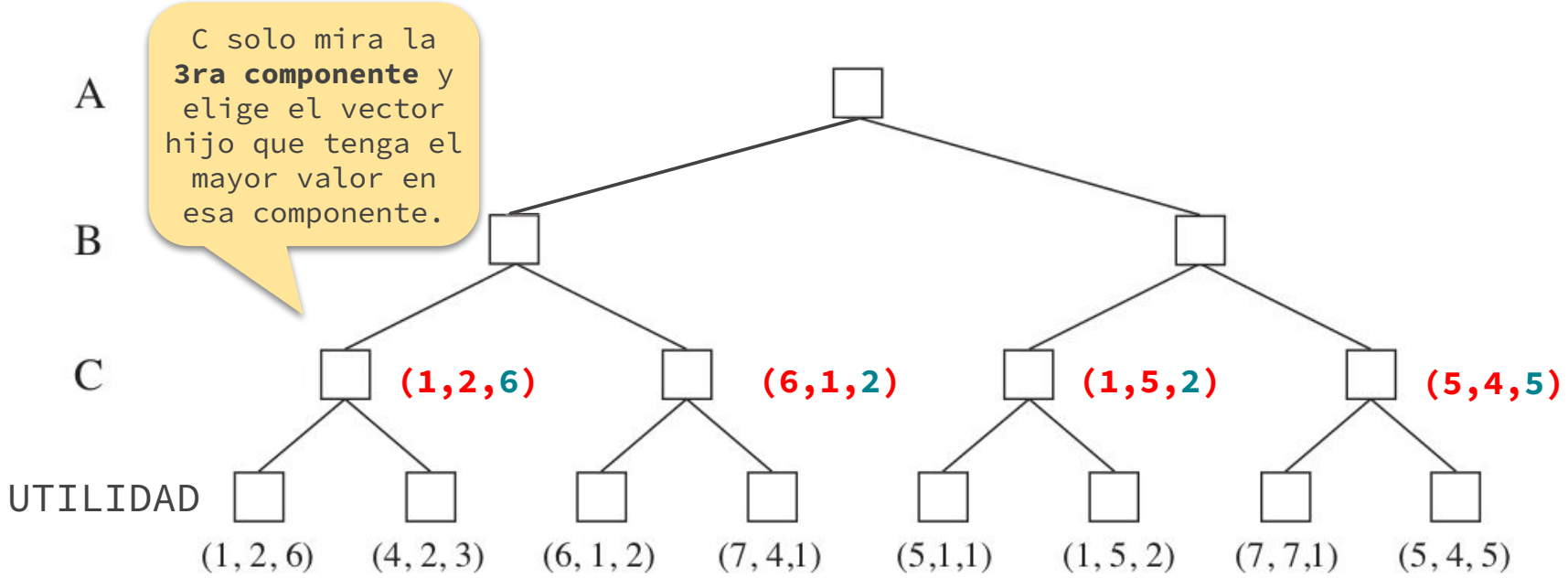
Árbol de juego



Valores minimax

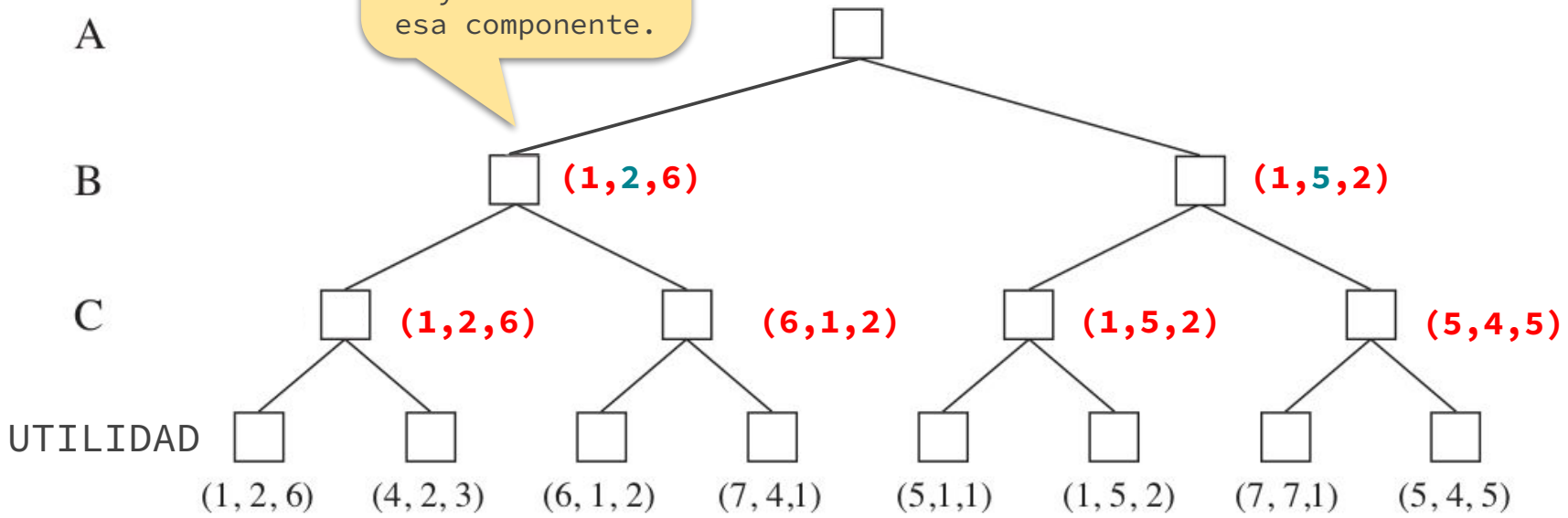
- El valor minimax de cada nodo es ahora un **vector de valores**.
- En los estados terminales, el valor minimax es la **utilidad**.
- En estados no terminales, el jugador que tiene el turno elegirá, entre los sucesores, aquel cuyo vector tenga **el mayor valor desde su propia perspectiva**.
- Veamos un ejemplo…

📖 Ejemplo


C solo mira la **3ra componente** y elige el vector hijo que tenga el mayor valor en esa componente.
A
B
C
UTILIDAD
(1,2,6)
(6,1,2)
(1,5,2)
(5,4,5)
(1, 2, 6)
(4, 2, 3)
(6, 1, 2)
(7, 4,1)
(5,1,1)
(1, 5, 2)
(7, 7,1)
(5, 4, 5)

📖 Ejemplo

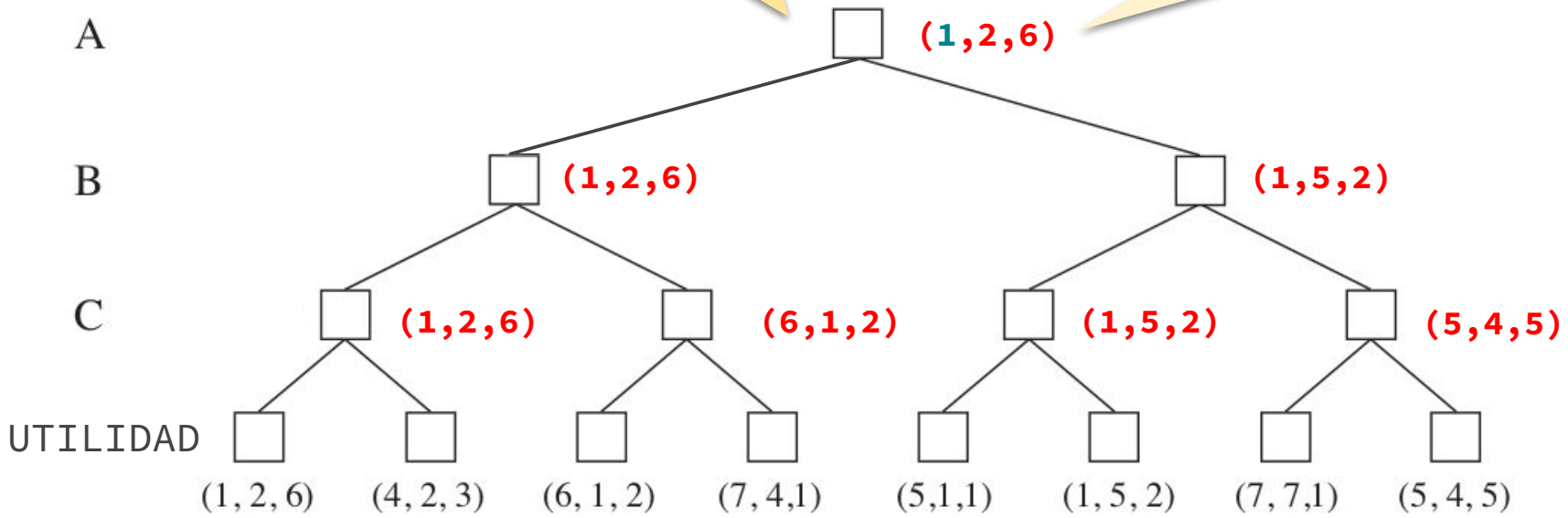
B solo mira la **2da componente** y elige el vector hijo que tenga el mayor valor en esa componente.



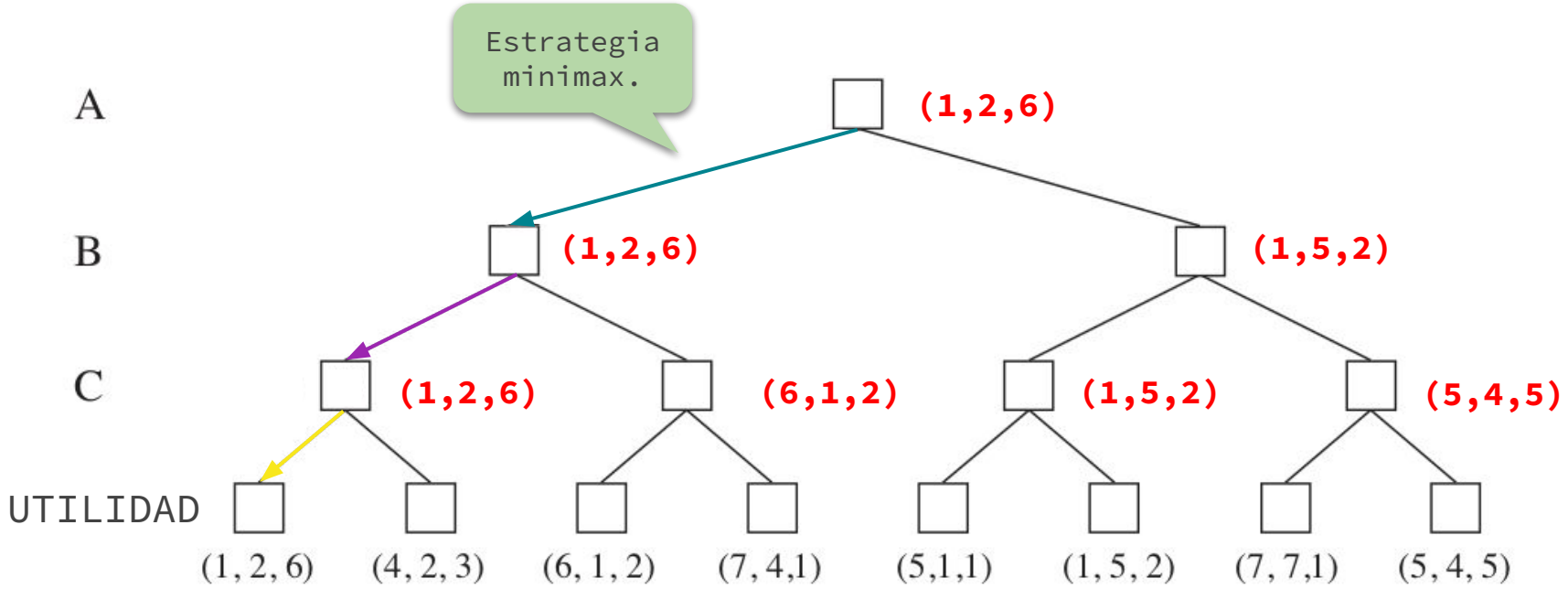
📖 Ejemplo

A solo mira la **1ra componente** y elige el vector hijo que tenga el mayor valor en esa componente.

También podría ser (1,5,2), pues hay empate en la 1ra componente.

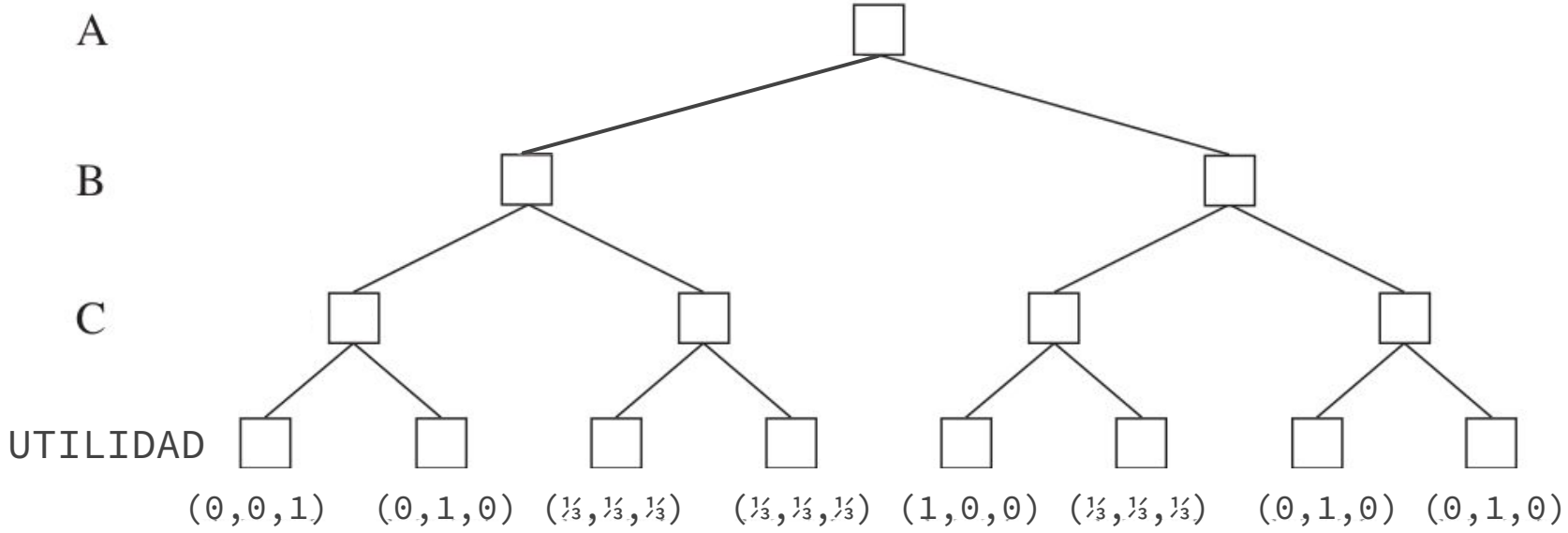


📖 Ejemplo

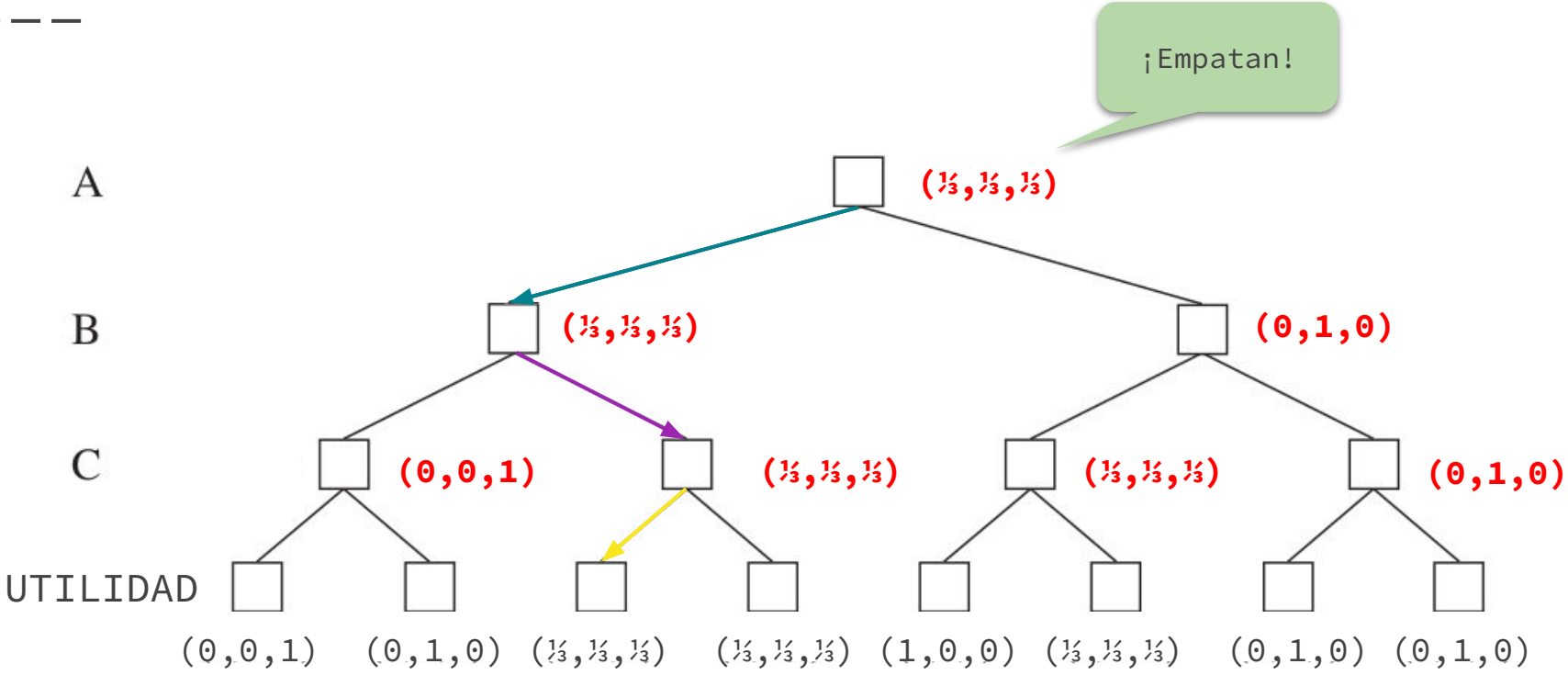

Estrategia minimax.
A
B
C
UTILIDAD
(1,2,6)
(1,2,6)
(1,5,2)
(1,2,6)
(6,1,2)
(1,5,2)
(5,4,5)
(1, 2, 6)
(4, 2, 3)
(6, 1, 2)
(7, 4,1)
(5,1,1)
(1, 5, 2)
(7, 7,1)
(5, 4, 5)

Ejercicio

¿Quién gana el siguiente juego suponiendo que todos juegan óptimamente?



Respuesta


¡Empatan!
A
B
C
UTILIDAD
(⅓,⅓,⅓)
(⅓,⅓,⅓)
(0,1,0)
(0,0,1)
(⅓,⅓,⅓)
(⅓,⅓,⅓)
(0,1,0)
(0,0,1)
(0,1,0)
(⅓,⅓,⅓)
(⅓,⅓,⅓)
(1,0,0)
(⅓,⅓,⅓)
(0,1,0)
(0,1,0)

Alianzas

- En los juegos multijugadores suelen producirse **alianzas**.
- Las alianzas se pueden **hacer** y **romper** a medida que el juego avanza.
- Las alianzas pueden ser consecuencias naturales de las estrategias óptimas de cada jugador.
- 📖 **Ejemplo.** Si A y B están en una posición más débil que C, puede ser óptimo para A y B atacar juntos a C en vez de atacarse entre ellos.

Juegos estocásticos

En los **juegos estocásticos** suceden eventos externos impredecibles.

Típicamente mediante elementos **aleatorios**, como por ejemplo una tirada de dados.

Combinan, en mayor o menor medida, **suerte** y **habilidad**.

Ta-te-ti probabilístico

En el ta-te-ti clásico, el resultado de aplicar una acción es escribir el símbolo del jugador actual (con 100% de probabilidad).

En el ta-te-ti probabilístico el resultado depende de las **probabilidades** dadas en cada casilla.

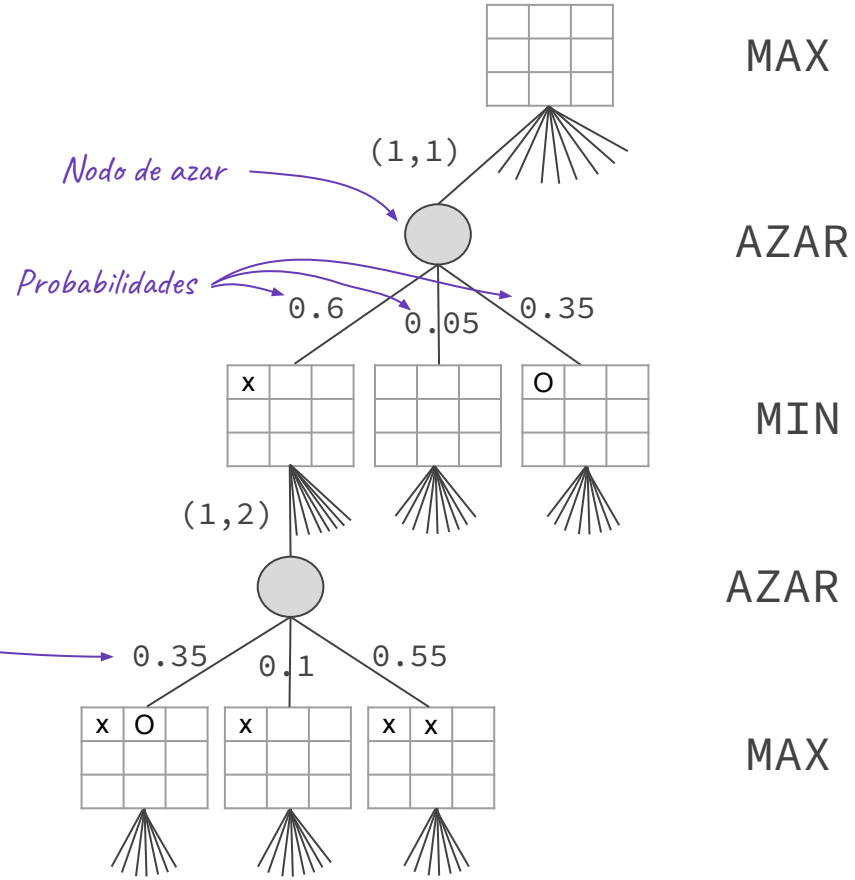
- 😊: Probabilidad de escribir **nuestro símbolo**.
- 😐: Probabilidad de no escribir **ningún símbolo**.
- 😞: Probabilidad de escribir el **símbolo del oponente**.

https://www.csun.io/2024/06/08/probabilistic-tic-tac-toe.html

Nodos de azar

- En juegos con azar, la definición de la función acciones o el modelo de transiciones suelen incluir **probabilidades**.
- No es posible construir un árbol de juego convencional.
- Se incorporan **nodos de azar** para representar los posibles eventos aleatorios.

Árbol de búsqueda



Minimax-esperado

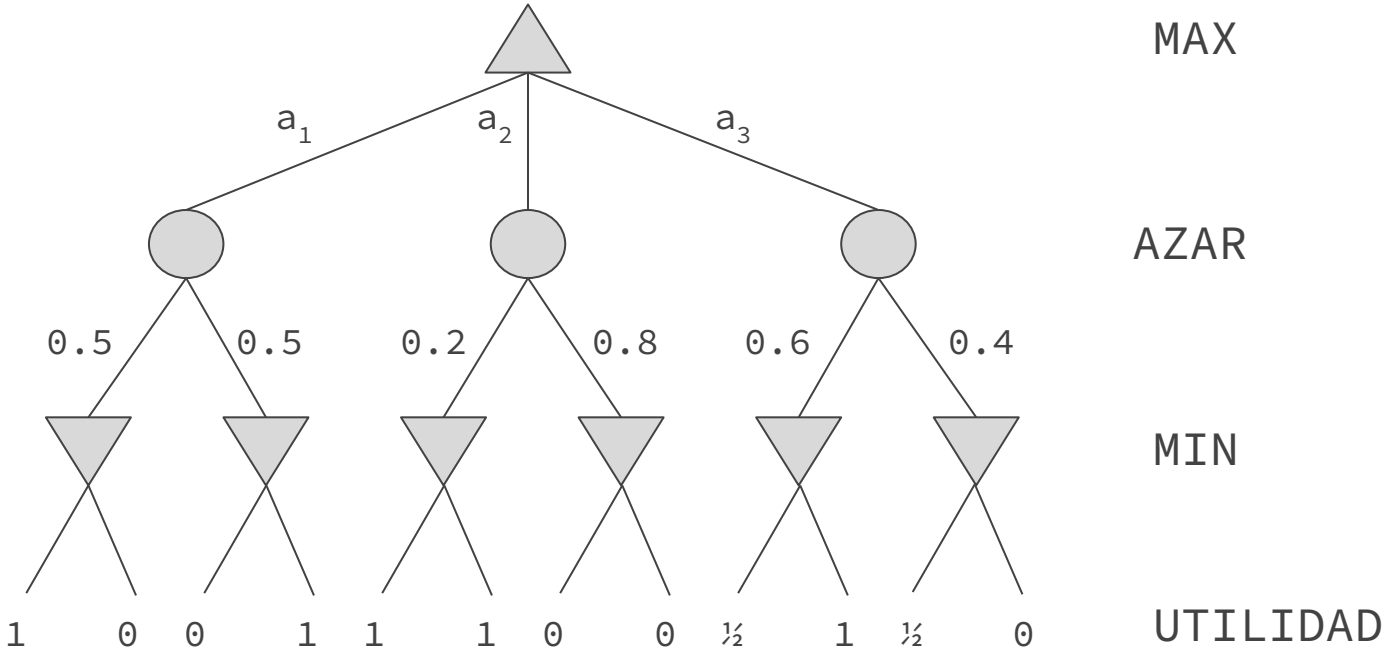
- ¿Cómo tomar decisiones correctas?
- Todavía buscamos la acción que nos lleve al mejor resultado.
- Los nodos ya **no tienen valores minimax exactos**.
- Podemos calcular el **valor esperado** en un nodo de azar: **promedio ponderado** entre todos los posibles resultados.
- Valor minimax → **Valor minimax-esperado** (expecti-minimax).

Minimax-esperado

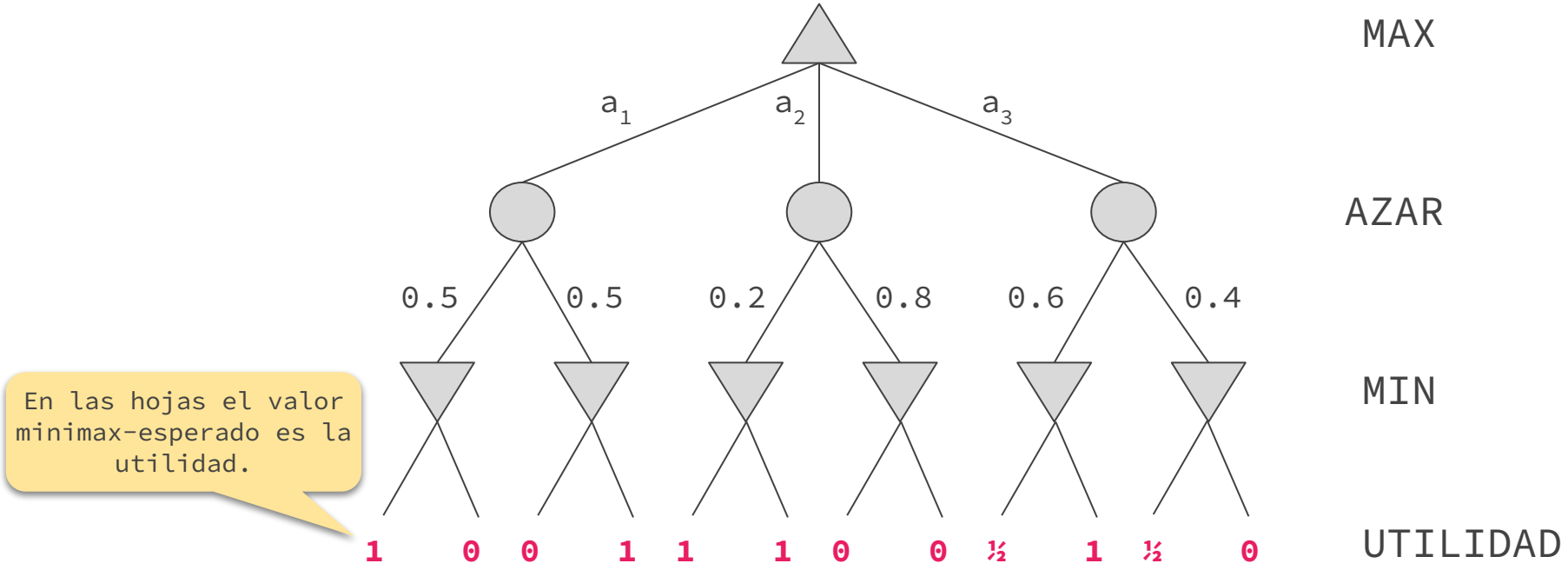
```
MINIMAX-ESPERADO(S) =
  { UTILIDAD(S,MAX)                                          SI TEST-TERMINAL(S)

    max {MINIMAX-ESPERADO(RESULTADO(S,A)):
         A ∈ ACCIONES(S)}                                    SI JUGADOR(S) = MAX

    min {MINIMAX-ESPERADO(RESULTADO(S,A)):
         A ∈ ACCIONES(S)}                                    SI JUGADOR(S) = MIN

    sum {P(R) * MINIMAX-ESPERADO(RESULTADO(S,R)):
        R ∈ ACCIONES(S)}                                     SI JUGADOR(S) = AZAR
```

R es el evento de azar y P(R) su probabilidad.
En el tateti probabilístico R ∈ {😊, 😐, 😞}

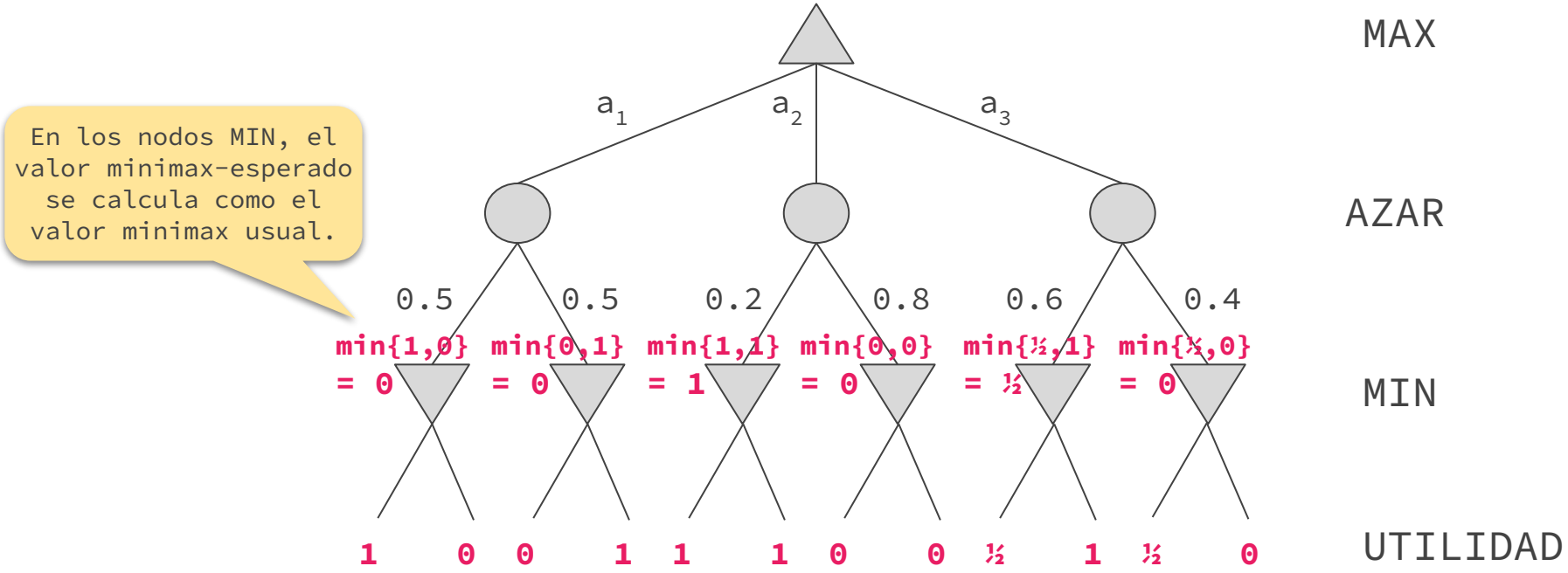
📖 Ejemplo



📖 Ejemplo

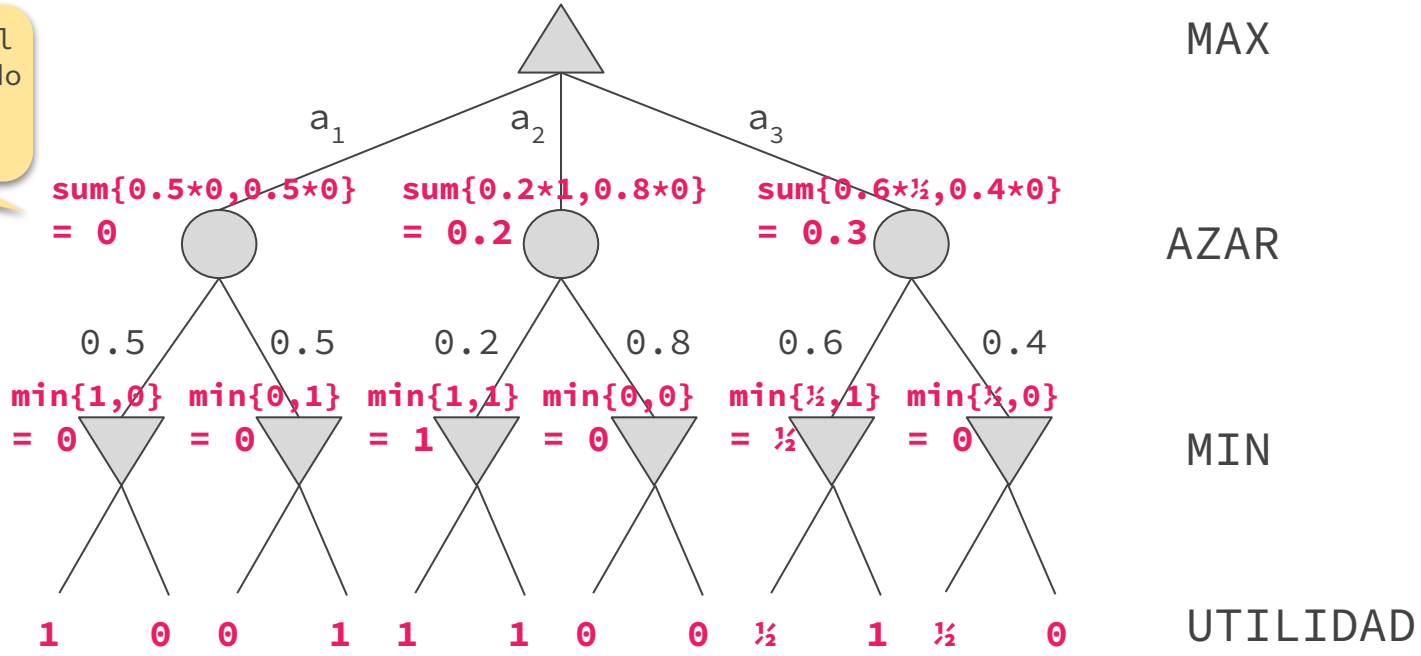

MAX
a_1
a_2
a_3
AZAR
0.5
0.5
0.2
0.8
0.6
0.4
MIN
UTILIDAD
1 0 0 1 1 1 0 0 ½ 1 ½ 0
En las hojas el valor minimax-esperado es la utilidad.

📖 Ejemplo

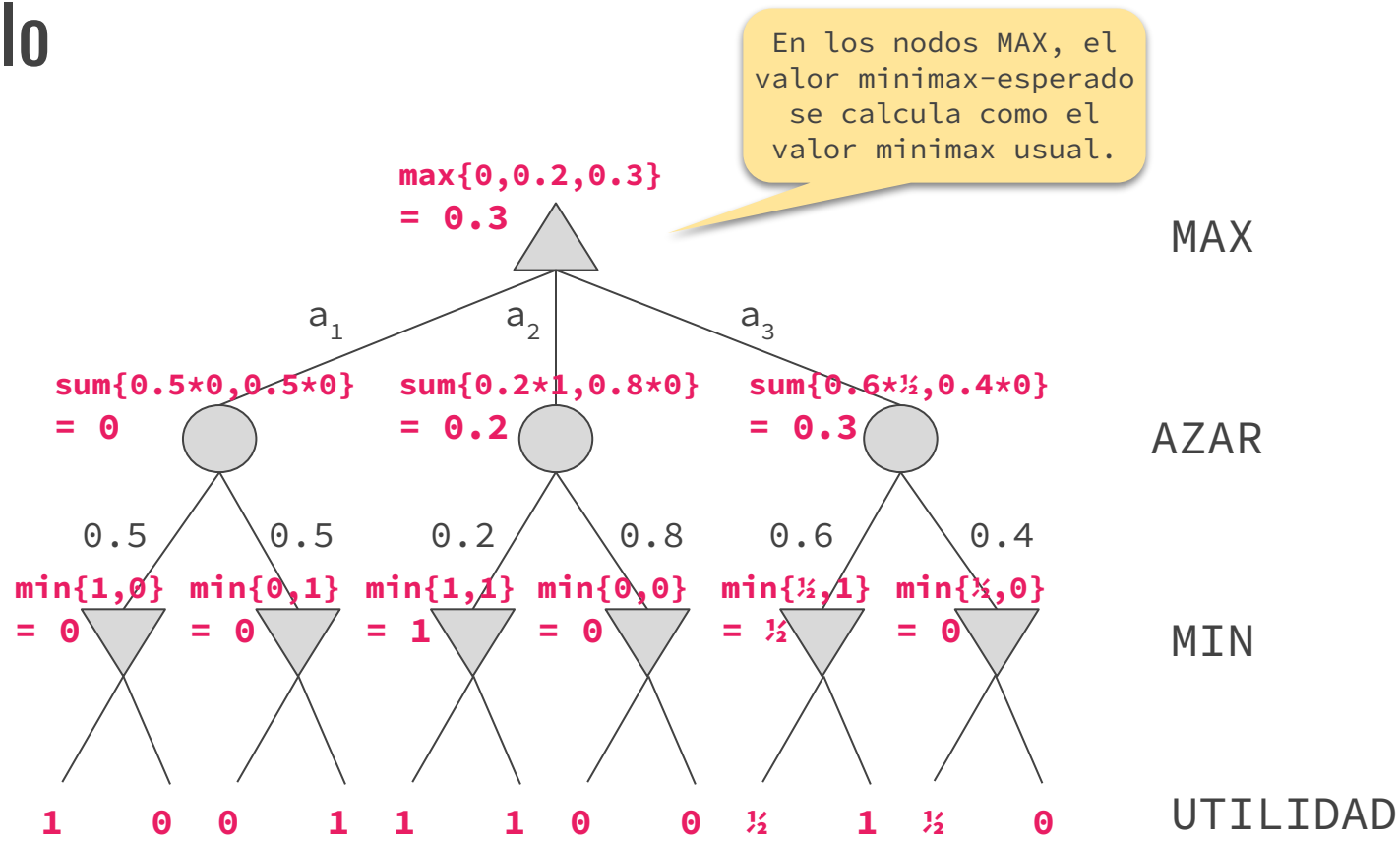

En los nodos MIN, el valor minimax-esperado se calcula como el valor minimax usual.
MAX
AZAR
MIN
UTILIDAD
a1
a2
a3
0.5
0.5
0.2
0.8
0.6
0.4
min{1,0} = 0
min{0,1} = 0
min{1,1} = 1
min{0,0} = 0
min{½,1} = ½
min{½,0} = 0
1 0 0 1 1 1 0 0 ½ 1 ½ 0

📖 Ejemplo

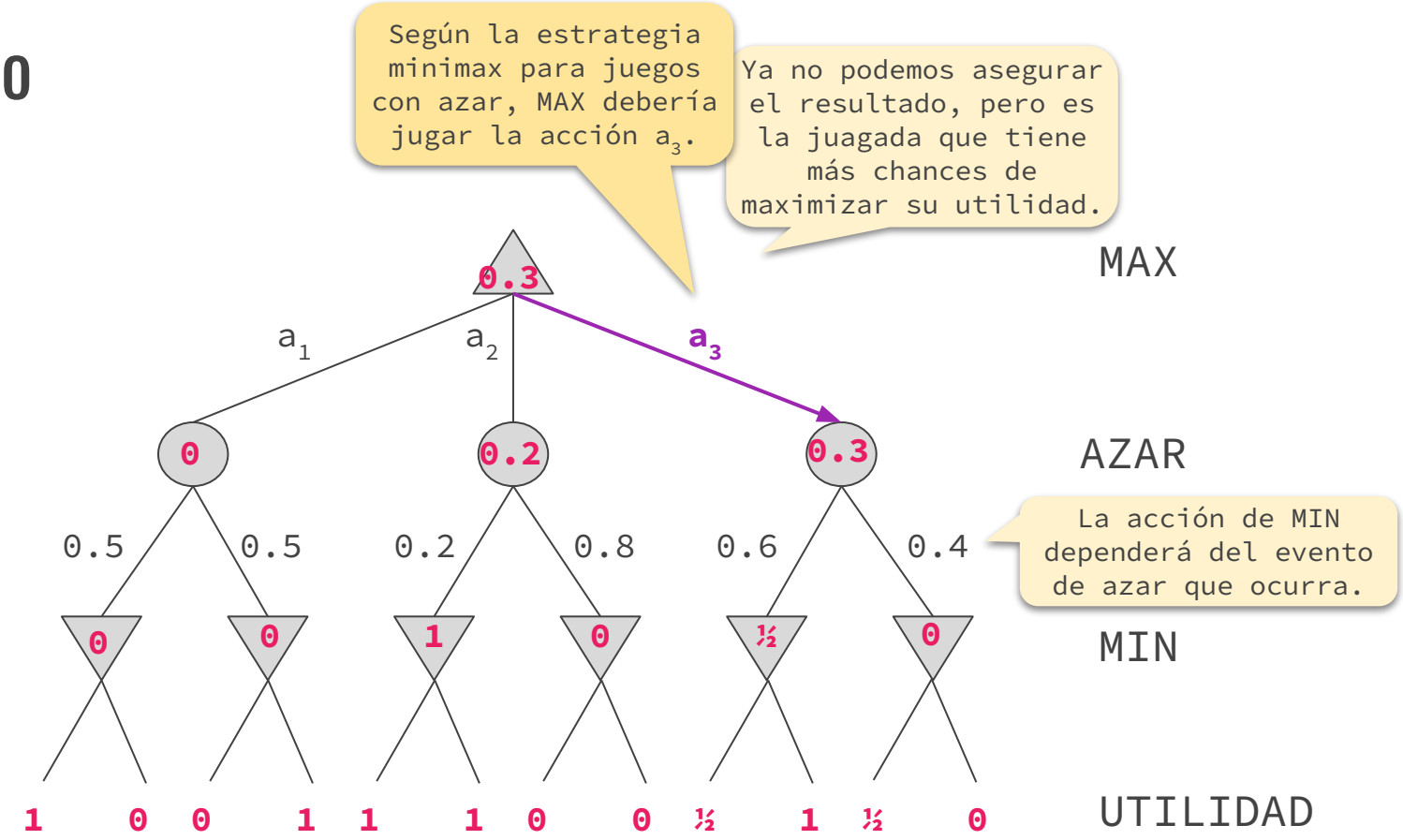
En los nodos AZAR, el valor minimax-esperado se calcula usando promedio ponderado.



📖 Ejemplo

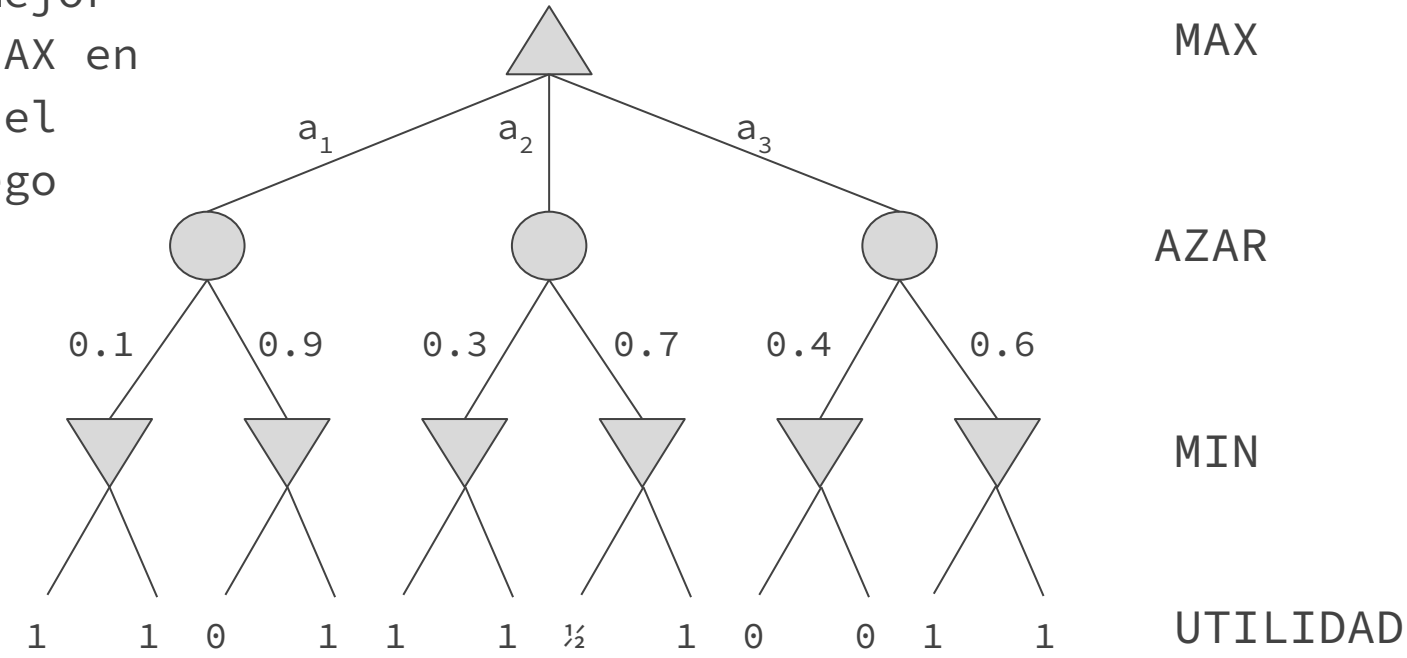

En los nodos MAX, el valor minimax-esperado se calcula como el valor minimax usual.
max{0,0.2,0.3}
= 0.3
MAX
a1
a2
a3
sum{0.5*0,0.5*0}
= 0
sum{0.2*1,0.8*0}
= 0.2
sum{0.6*½,0.4*0}
= 0.3
AZAR
0.5
0.5
0.2
0.8
0.6
0.4
min{1,0}
= 0
min{0,1}
= 0
min{1,1}
= 1
min{0,0}
= 0
min{½,1}
= ½
min{½,0}
= 0
MIN
1
0
0
1
1
1
0
0
½
1
½
0
UTILIDAD

📖 Ejemplo

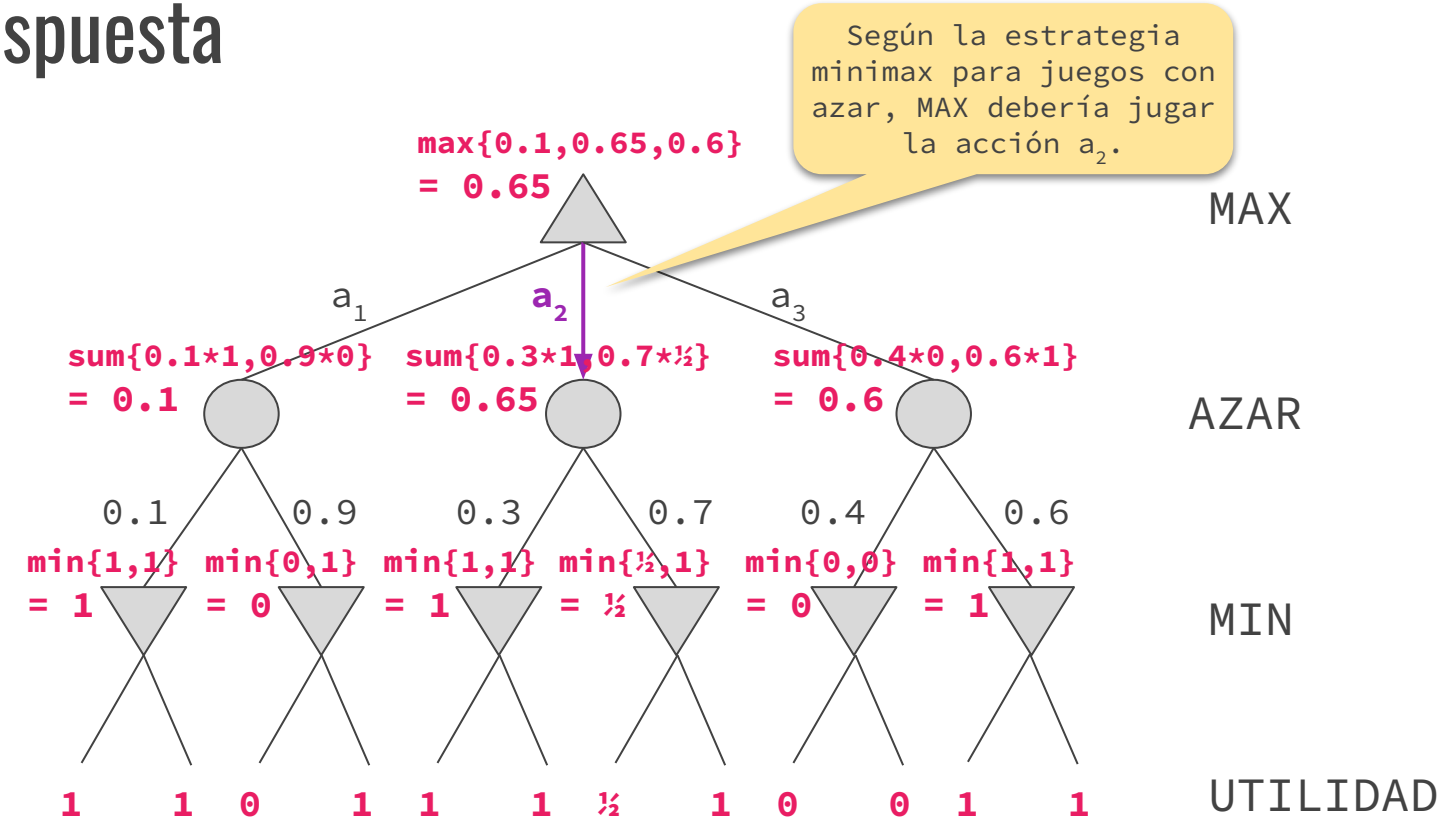

Según la estrategia minimax para juegos con azar, MAX debería jugar la acción a_3.
Ya no podemos asegurar el resultado, pero es la juagada que tiene más chances de maximizar su utilidad.
MAX
0.3
a_1
a_2
a_3
AZAR
0
0.2
0.3
0.5
0.5
0.2
0.8
0.6
0.4
La acción de MIN dependerá del evento de azar que ocurra.
MIN
0
0
1
0
½
0
UTILIDAD
1 0 0 1 1 1 0 0 ½ 1 ½ 0

Ejercicio

¿Cuál es la mejor acción para MAX en la apertura del siguiente juego con azar?



Respuesta


Según la estrategia minimax para juegos con azar, MAX debería jugar la acción a_2.
max{0.1,0.65,0.6}
= 0.65
MAX
a_1
a_2
a_3
sum{0.1*1,0.9*0}
= 0.1
sum{0.3*1,0.7*½}
= 0.65
sum{0.4*0,0.6*1}
= 0.6
AZAR
0.1
0.9
0.3
0.7
0.4
0.6
min{1,1}
= 1
min{0,1}
= 0
min{1,1}
= 1
min{½,1}
= ½
min{0,0}
= 0
min{1,1}
= 1
MIN
1 1 0 1 1 1 ½ 1 0 0 1 1
UTILIDAD

Performance de minimax-esperado

- Si el programa supiera de antemano los eventos aleatorios que ocurrirán, resolver el juego sería equivalente a resolver un juego sin azar.
- Minimax genera a lo sumo b^{m+1} nodos, siendo b el factor de ramificación y m es la altura del árbol anterior.
- Minimax-esperado considera **todos los posibles eventos aleatorios**.

🚩 Minimax-esperado genera a lo sumo $(b.n)^{m+1}$, siendo n el número de eventos posibles.

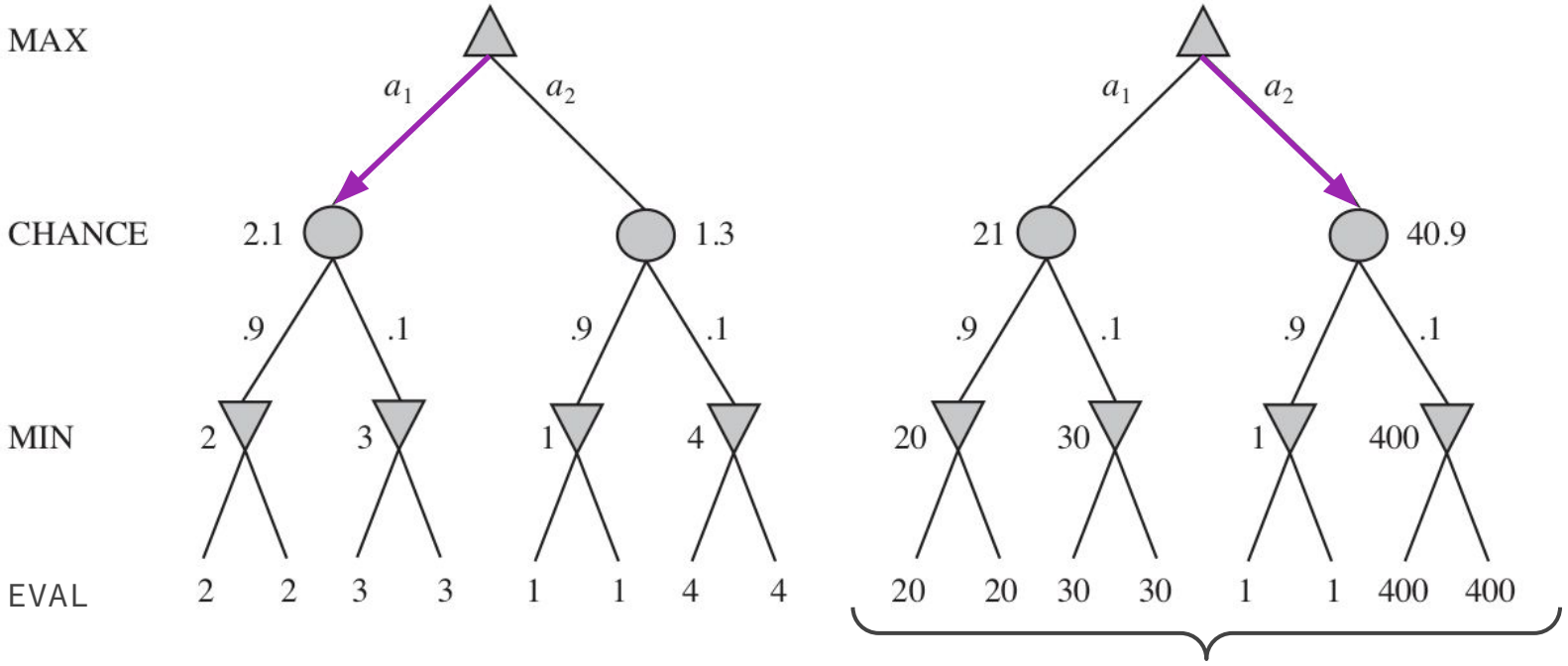
Performance de minimax-esperado

- En la mayoría de los juegos de azar, hay muchos eventos aleatorios.
  - 📖 **Ejemplo.** Una tirada de un dado involucra $n = 6$.
- El mayor consumo de tiempo de minimax-esperado comparado al de minimax vuelve **poco realista considerar el árbol de juego completo**.
- En la práctica, se introducen un **test de corte** y una **función de evaluación**, al igual que en minimax.
- Incluso así, no es esperable que podamos mirar demasiado hacia adelante, **solo unos pocos niveles**.

Función de evaluación

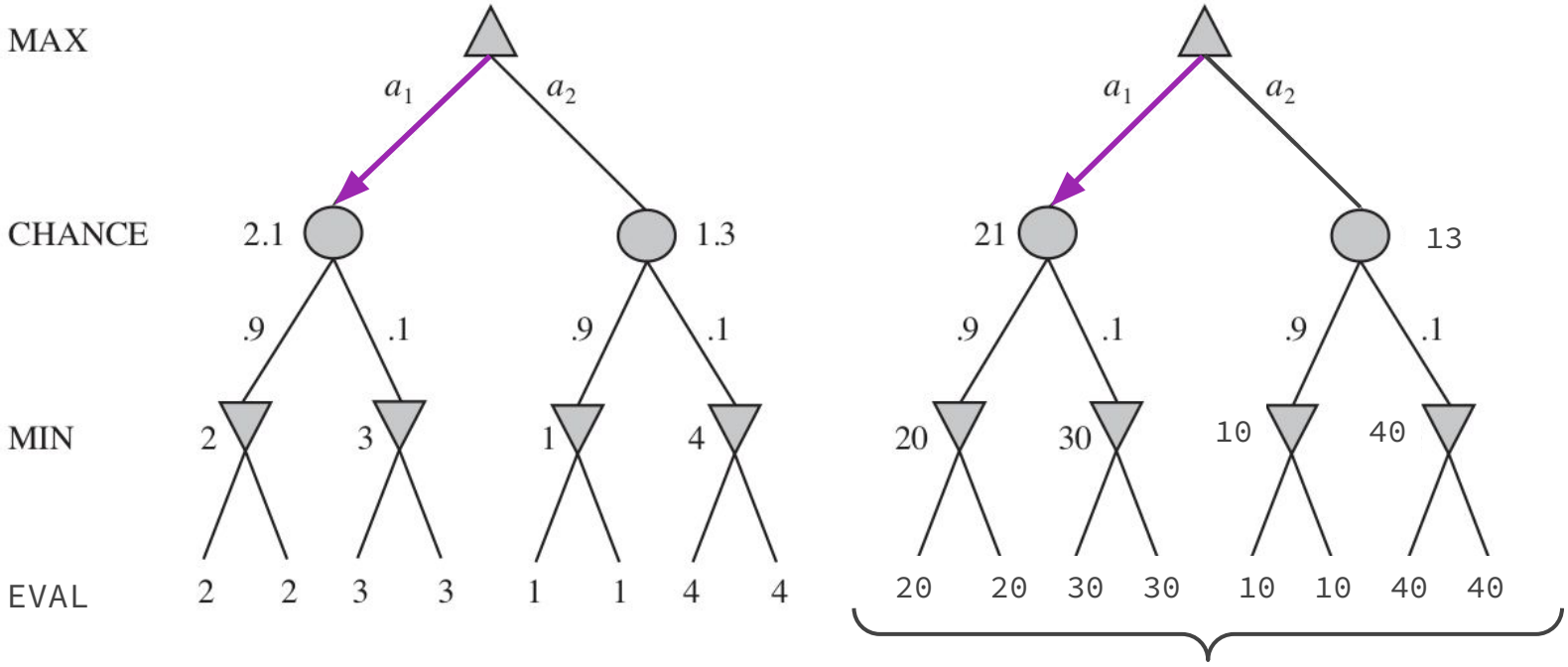
- En minimax, una buena función de evaluación necesita asignar **mayores valores a resultados más prometedores.**
- En precedencia de nodos de azar, hay que ser más **cuidadosos** con las estimaciones.
- Veamos un ejemplo…

Funciones de evaluación



🚨 Cambiando los valores, pero preservando el orden, es posible **alterar la decisión final** 🚨

Funciones de evaluación



Para evitar esta sensibilidad, la función de evaluación tiene que ser una **transformación lineal.**

Poda alfa-beta

En determinadas circunstancias, es posible computar la estrategia minimax **sin mirar todos** los nodos del árbol de juego.

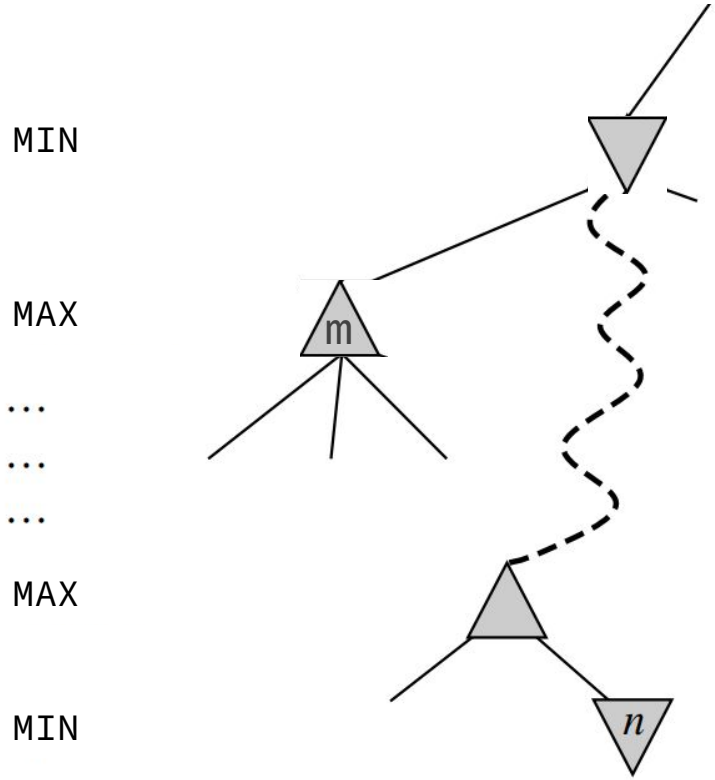
Basta con **podar** aquellos nodos para los cuales se tenga evidencia suficiente de que **no influyen** en la decisión final.

Esta técnica se llama **poda alfa-beta**.

Poda alfa-beta

Si MAX tiene una opción *n*, pero más arriba en el árbol MIN tiene una opción *m* mejor (menor valor) que *n*, entonces *n* **nunca será alcanzado en una jugada real** (siempre asumiendo que MAX y MIN juegan óptimamente).

Si se tiene suficiente información sobre *m* y *n* para llegar a esta conclusión, entonces es posible podar nodos.



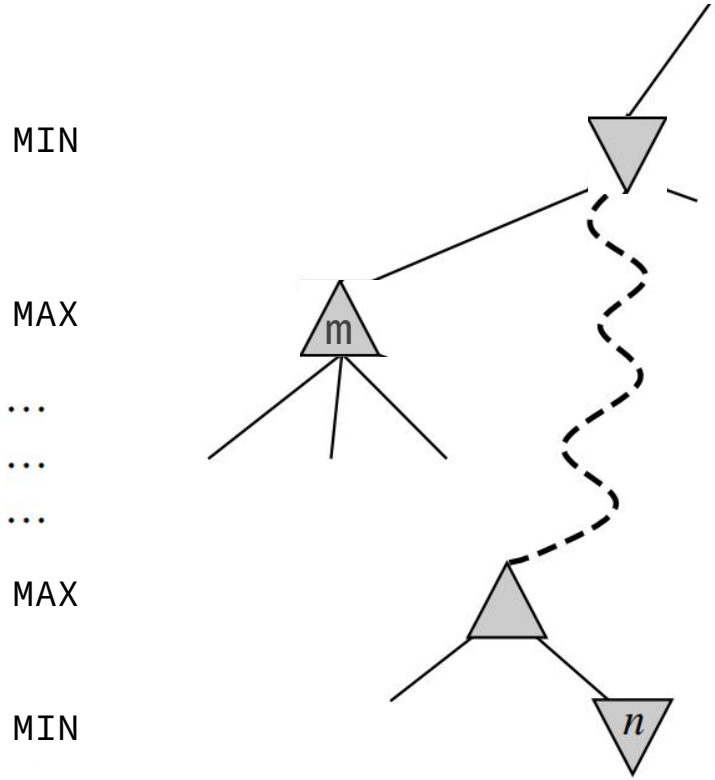
Poda alfa-beta

Supongamos que:

1. Computamos el valor minimax de *m* y es un número β. Es decir, MIN tiene una opción de valor β.

2. Más tarde, mientras computamos el valor minimax de *n*, logramos concluir que es mayor a β.

Entonces, podemos dejar de computar el valor minimax del padre de n **de inmediato**, es decir, **podando** los hijos que queden por calcular.

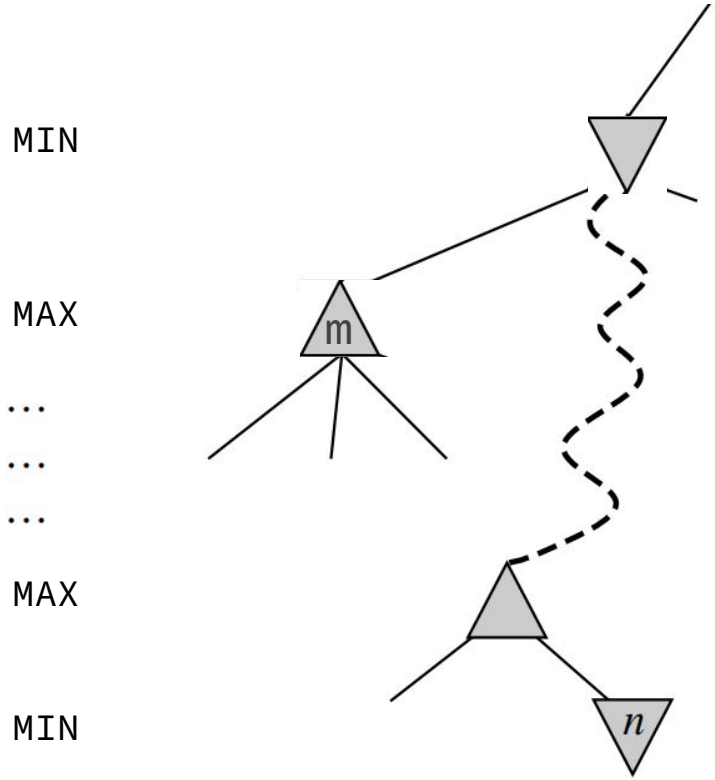


Poda alfa-beta

Durante la ejecución, se mantienen dos parámetros (α,β).

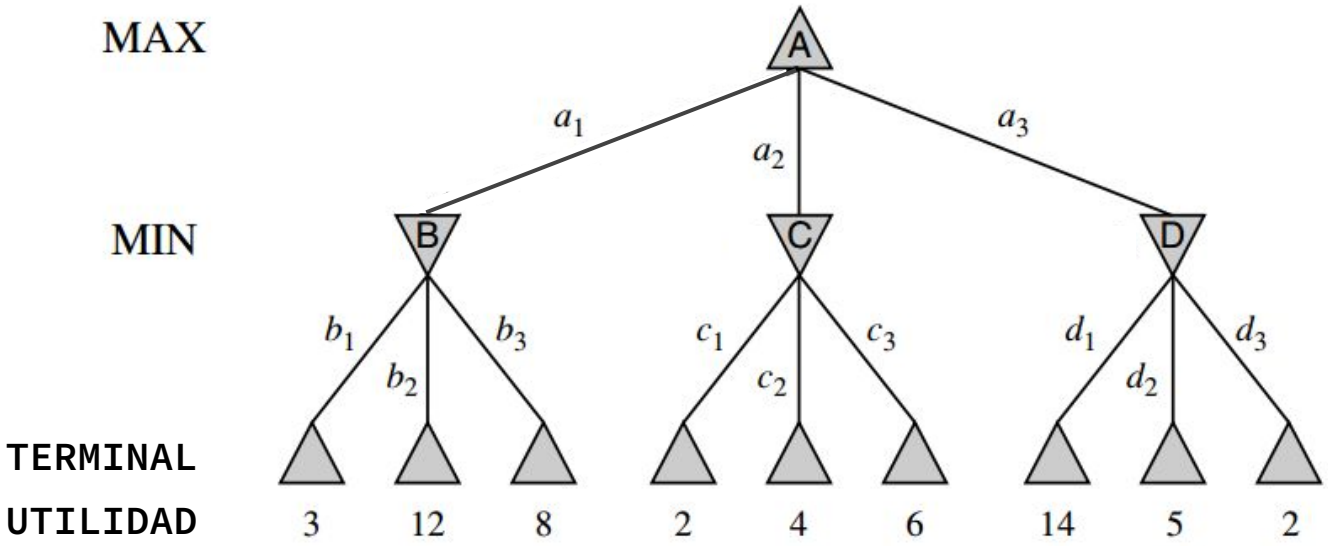
α: valor de la mejor decisión (mayor valor) encontrada hasta ahora en el camino para MAX.

β: valor de la mejor decisión (menor valor) encontrada hasta ahora en el camino para MIN.

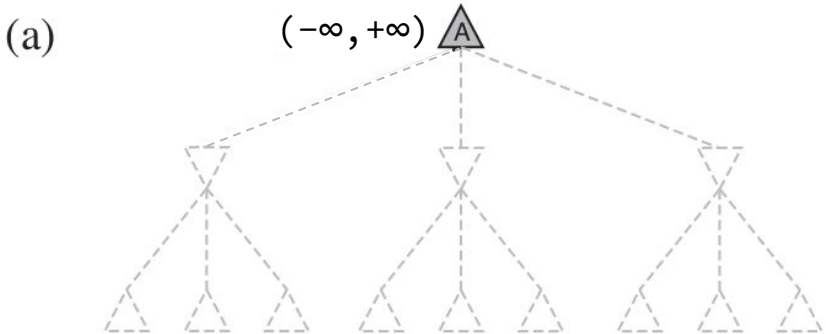


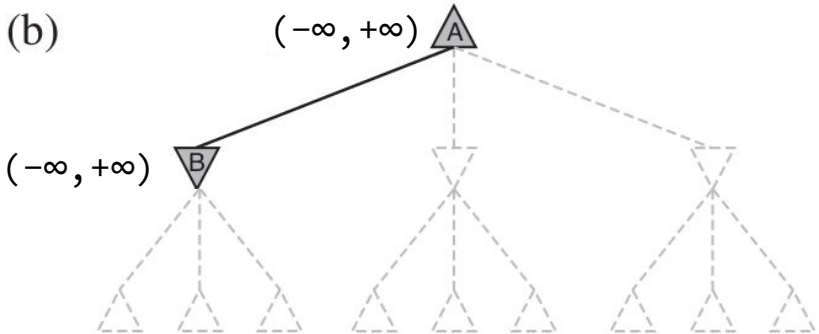
📖 Ejemplo

Vamos a aplicar el algoritmo MINIMAX con poda alfa-beta sobre el siguiente árbol de juego.



📖 Ejemplo





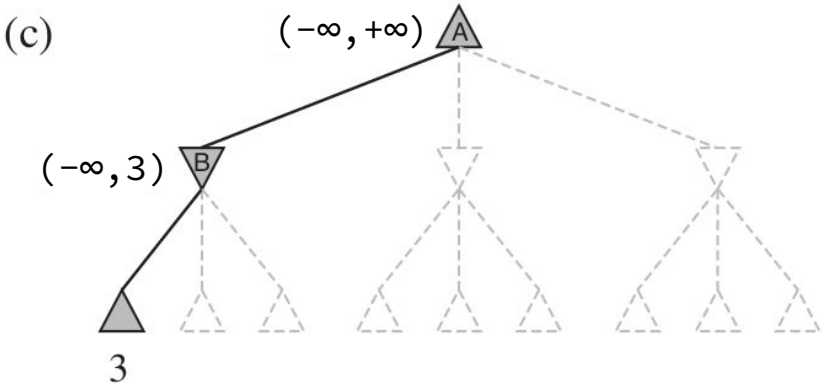
(a) Al inicio, como no hay información las cotas son (-∞,+∞).

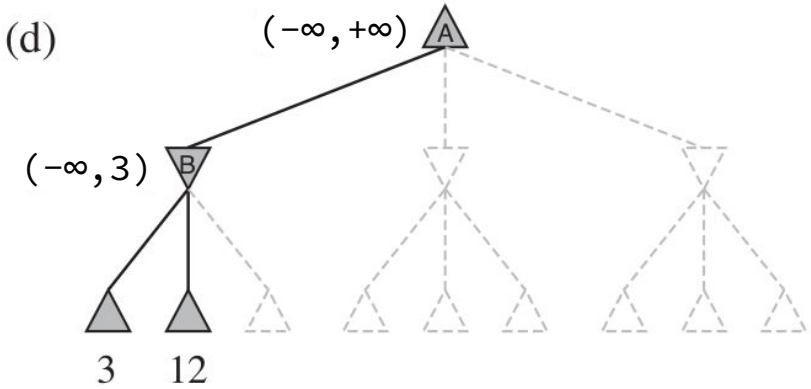
Para calcular el valor minimax de A, hay que conocer el de sus hijos, luego la búsqueda desciende.

(b) Como todavía no hay información las cotas siguen siendo (-∞,+∞).

Para calcular el valor minimax de B, hay que conocer el de sus hijos, luego la búsqueda desciende.

📖 Ejemplo





(c) La 1ra hoja debajo de B tiene valor 3.

La búsqueda retrocede a B. Hasta ahora, la mejor opción para MIN tiene valor β = 3, pero podría ser menor.

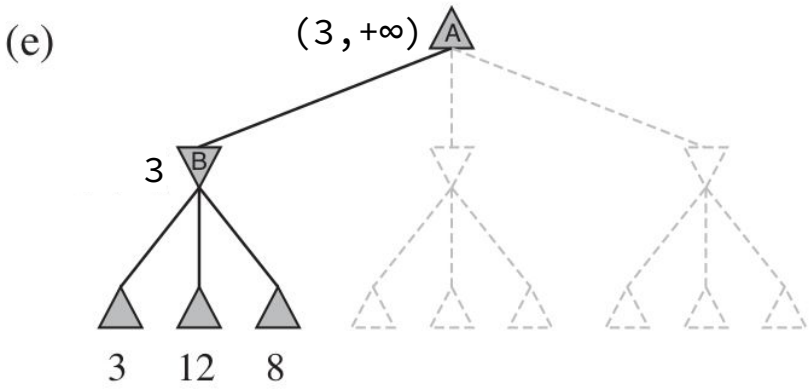
La búsqueda desciende hacia su próximo hijo.

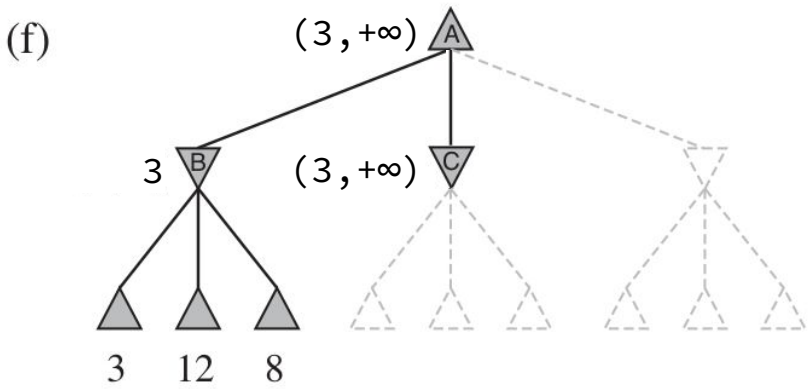
(d) La 2da hoja debajo de B tiene valor 12.

La búsqueda retrocede a B. Hasta ahora, la mejor opción para MIN sigue siendo la de valor β = 3.

La búsqueda desciende hacia su último hijo.

📖 Ejemplo





(e) La 3ra hoja debajo de B tiene valor 8.

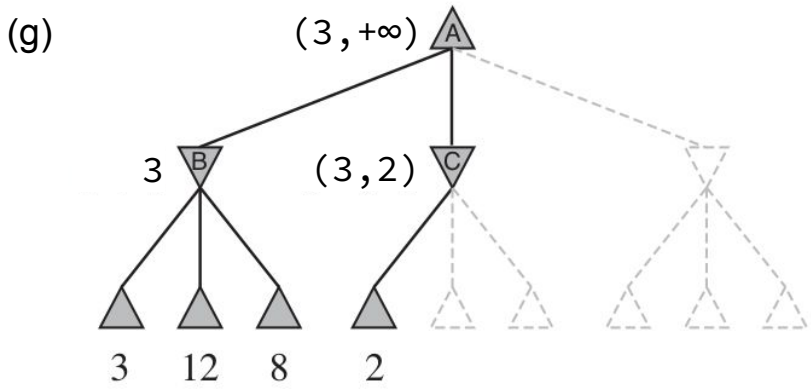
La búsqueda retrocede a B. Como todos sus sucesores fueron explorados, el valor minimax de B es exáctamente 3.

La búsqueda retrocede a A. Hasta ahora, la mejor opción para MAX tiene valor α = 3. La búsqueda desciende hacia su próximo hijo.

(f) Para calcular el valor minimax de C, hay que conocer el de sus hijos, luego la búsqueda desciende.

Se mantiene la cota α = 3, para indicar que MAX tiene una opción de valor 3 más arriba en el camino (en la raíz).

📖 Ejemplo



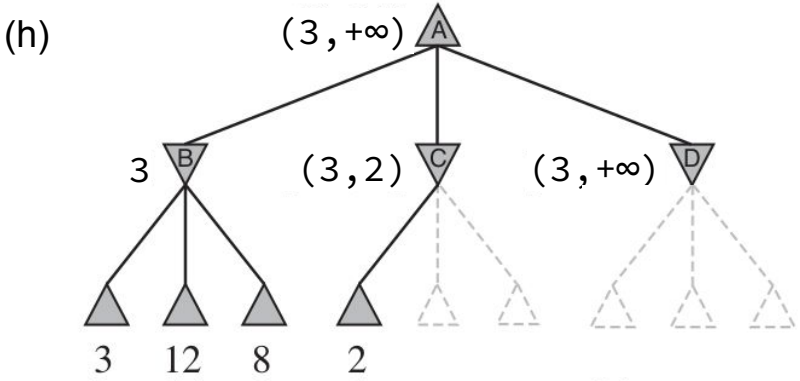
(g) La primera hoja debajo de C tiene valor 2.

La búsqueda retrocede a C. Hasta ahora, la mejor opción para MIN (en este camino) tiene valor β = 2, pero podría ser menor.

Sabemos que MAX tiene una opción de valor α = 3, luego MAX nunca elegirá a C porque su utilidad sería a lo sumo 2. Entonces podemos **podar** los demás hijos de C.

La búsqueda retrocede a A y desciende hacia su último hijo.

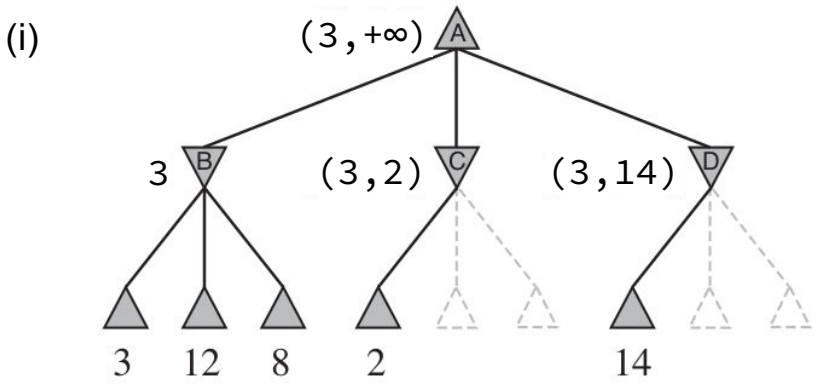
📖 Ejemplo



(h) Para calcular el valor minimax de D, hay que conocer el de sus hijos, luego la búsqueda desciende.

Se mantiene la cota α = 3, para indicar que MAX tiene una opción de valor 3 más arriba en el camino (en la raíz).

📖 Ejemplo

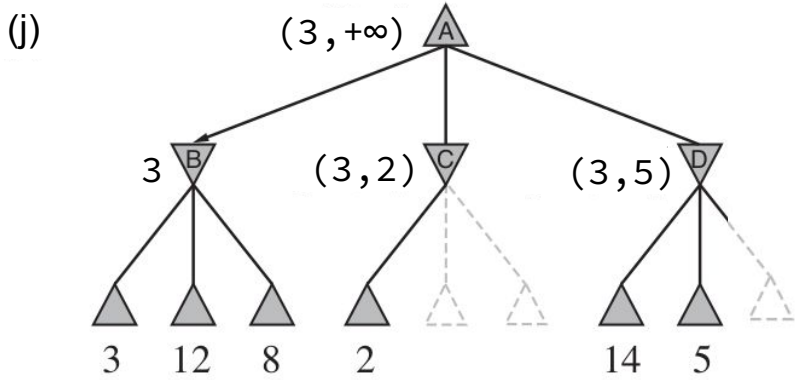


(i) La 1ra hoja debajo de D tiene valor 14.

La búsqueda retrocede a D. Hasta ahora, la mejor opción para MIN (en este camino) tiene valor β = 14, pero podría ser menor.

Dado que 14 > 3, todavía hay chances de mejorar la decisión de MAX en este camino, luego D no puede ser podado. La búsqueda desciende a su próximo hijo.

📖 Ejemplo

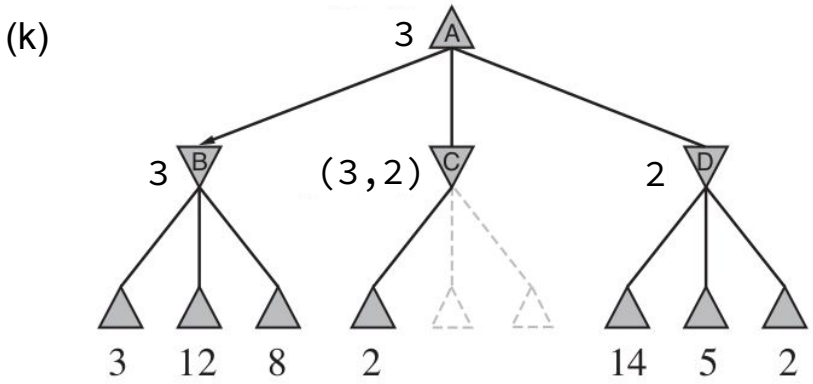


(j) La 2da hoja debajo de D tiene valor 5.

La búsqueda retrocede a D. Hasta ahora, la mejor opción para MIN (en este camino) tiene valor β = 5, pero podría ser menor.

Dado que 5 > 3, todavía hay chances de mejorar la decisión de MAX en este camino, luego D no puede ser podado. La búsqueda desciende a su último hijo.

📖 Ejemplo



La poda alfa-beta evitó mirar dos nodos del árbol de juego.

(j) La 3ra hoja debajo de D tiene valor 2.

La búsqueda retrocede a D. Como todos sus sucesores fueron explorados, el valor minimax de D es exáctamente 2.

La búsqueda retrocede a A. Como todos sus sucesores fueron explorados, el valor minimax de A es exáctamente 3. Por lo tanto, la mejor decisión para MAX es moverse a B.

📖 Ejemplo

Otra forma de verlo…

$$
\begin{aligned}
\textsc{Minimax-Value}(\textit{raíz}) &= \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2)) \\
&= \max(3, \min(2, x, y), 2) \\
&= \max(3, z, 2) \qquad \text{donde } z \leq 2 \\
&= 3
\end{aligned}
$$

donde x e y son los valores minimax de los hijos podados de C. En otras palabras, **la decisión minimax es independiente de las hojas podadas**.

Poda alfa-beta

En general,

- Se puede aplicar a árboles de cualquier altura.
- Puede podar subárboles enteros y no solo hojas.

Implementación de la poda alfa beta

- Las funciones MINIMAX-MAX y MINIMAX-MIN se aumentan con dos nuevos parámetros: α y β.
- MINIMAX-MAX:
  - Actualiza α si el valor minimax del nodo es más grande que α.
  - Termina la llamada recursiva si un hijo tiene valor minimax más grande que β.
- MINIMAX-MIN:
  - Actualiza β si el valor minimax del nodo es más chico que β.
  - Termina la llamada recursiva si un hijo tiene valor minimax más chico que α.

Algoritmo minimax con poda alfa-beta

```
# Calcula recursivamente el valor minimax
# de un nodo MAX con poda alfa-beta

function MINIMAX-MAX(problema,estado,α,β) return valor
    if problema.TEST-TERMINAL(estado) then
        return problema.UTILIDAD(estado,MAX)
    valor ← -∞
    forall acción in problema.ACCIONES(estado) do
        sucesor ← problema.RESULTADO(estado, acción)
        valor ← max(valor,
                    MINIMAX-MIN(problema,sucesor,α,β))
        if valor ≥ β then return valor # fin llamada
        α ← max(α,valor)
    return valor
```

```
# Calcula recursivamente el valor minimax
# de un nodo MIN con poda alfa-beta

function MINIMAX-MIN(problema,estado,α,β) return valor
    if problema.TEST-TERMINAL(estado) then
        return problema.UTILIDAD(estado,MAX)
    valor ← +∞
    forall acción in problema.ACCIONES(estado) do
        sucesor ← problema.RESULTADO(estado, acción)
        valor ← min(valor,
                    MINIMAX-MAX(problema,sucesor,α,β))
        if valor ≤ α then return valor # fin llamada
        β ← min(β,valor)
    return valor
```

💻 Algoritmo minimax con poda alfa-beta

Algoritmo que implementa la estrategia minimax con poda alfa-beta. Toma un estado y devuelve la acción a aplicar en ese estado.

```
function MINIMAX-ALFA-BETA(problema, estado) return acción
  if problema.JUGADOR(estado) == MAX:
      sucs ← {acción: MINIMAX-MIN(problema, problema.RESULTADO(estado, acción), -∞, +∞)
              for acción in problema.ACCIONES(estado)}
      return max(sucs, key=sucs.get) # obtiene la clave con el mayor valor
  if problema.JUGADOR(estado) == MIN:
      sucs ← {acción: MINIMAX-MAX(problema, problema.RESULTADO(estado, acción), -∞, +∞)
              for acción in problema.ACCIONES(estado)}
      return min(sucs, key=sucs.get) # obtiene la clave con el mayor valor
```

Performance de MINIMAX-ALFA-BETA

- Devuelve lo mismo que MINIMAX sin poda. Es decir, la estrategia óptima.
- Menor consumo de tiempo que MINIMAX sin poda, aunque sigue siendo exponencial en la altura.
- Es muy dependiente del orden en el que se examinan los sucesores… No veremos mayores detalles.

➡️ Próximamente

Más allá de la búsqueda clásica...

Veremos algoritmos de **búsqueda local**: evalúan y modifican uno o varios estados actuales, en lugar de explorar sistemáticamente caminos desde un estado inicial